DATA ANALYST PROJECT · PD06

Target Dataset Exploration

— SQL & Python Analytics

An end-to-end analytical project combining PostgreSQL SQL querying with Python and Pandas-based analysis and visualization across customers, orders, products, sellers, revenue, retention, and sales trends.

PostgreSQL | SQL | Python | Pandas | Matplotlib | Google Colab

PROJECT STRUCTURE **15 analytical problems** Basic → Intermediate → Advanced

05
BASIC
Foundational SQL & Python analysis

05
INTERMEDIATE
CTEs, aggregation & correlation

05
ADVANCED
Window functions & retention

DATASET SCALE

99,441 CUSTOMERS	99,441 ORDERS
112,650 ORDER ITEMS	103,886 PAYMENTS
32,951 PRODUCTS	3,095 SELLERS

ANALYTICAL FOCUS Customer & sales analysis · revenue · ranking · time series · retention

QUESTION 01 / 15

List all unique cities where customers are located.

BASIC ANALYTICS

QUERY.SQL · POSTGRESQL

```sql
-- List all unique cities where customers are located.

SELECT DISTINCT customer_city
FROM customers;
```

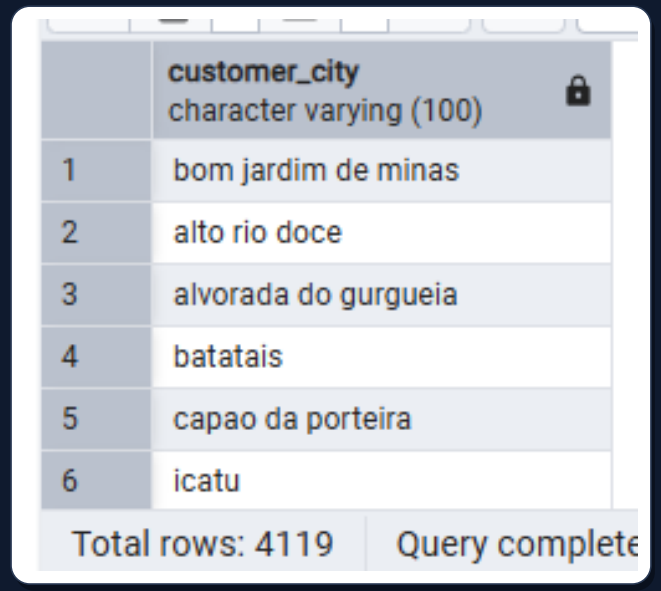
ANALYSIS.PY · PANDAS

```python
# BASIC PROBLEM 1
# List all unique cities where customers are located.

unique_cities = customers["customer_city"].dropna().unique()

print("Total unique cities:", len(unique_cities))
print("\nUnique cities:")
print(unique_cities)
```

SQL OUTPUT



	customer_city character varying (100)
1	bom jardim de minas
2	alto rio doce
3	alvorada do gurgueia
4	batatais
5	capao da porteira
6	icatu

Total rows: 4119 | Query complete

PYTHON OUTPUT

```
Total unique cities: 4119

Unique cities:
['franca' 'sao bernardo do campo' 'sao paulo' ... 'monte bonito'
 'sao rafael' 'eugenio de castro']
```

Source-faithful analytical code shown; visualization output is preserved from the notebook.

QUESTION 02 / 15

Count the number of orders placed in 2017.

BASIC ANALYTICS

QUERY.SQL · POSTGRESQL

```sql
-- Count the number of orders placed in 2017.

SELECT COUNT(*) AS total_orders_2017
FROM orders
WHERE order_purchase_timestamp >= '2017-01-01'
  AND order_purchase_timestamp < '2018-01-01';
```

ANALYSIS.PY · PANDAS

```python
# BASIC PROBLEM 2
# Count the number of orders placed in 2017.

orders_2017 = orders[
    (orders["order_purchase_timestamp"] >= "2017-01-01") &
    (orders["order_purchase_timestamp"] < "2018-01-01")
]

total_orders_2017 = len(orders_2017)

print("Total orders placed in 2017:", total_orders_2017)
```

SQL OUTPUT

	total_orders_2017 bigint
1	45101

PYTHON OUTPUT / VISUALIZATION



Source-faithful analytical code shown; visualization output is preserved from the notebook.

Find the total sales per category.

QUERY.SQL

```sql
-- Find the total sales per category.

SELECT
    p."product category" AS product_category,
    SUM(oi.price) AS total_sales
FROM order_items oi
JOIN products p
    ON oi.product_id = p.product_id
GROUP BY p."product category"
ORDER BY total_sales DESC;
```

ANALYSIS.PY

```python
# BASIC PROBLEM 3
# Find the total sales per category.

# Calculate total sales by product category
sales_by_category = (
    order_items
    .merge(
        products[["product_id", "product category"]],
        on="product_id",
        how="inner"
    )
    .groupby("product category", as_index=False)["price"]
    .sum()
    .rename(columns={
        "product category": "product_category",
        "price": "total_sales"
    })
    .sort_values("total_sales", ascending=False)
)

# Select top 8 categories
top_8_categories = sales_by_category.head(8)

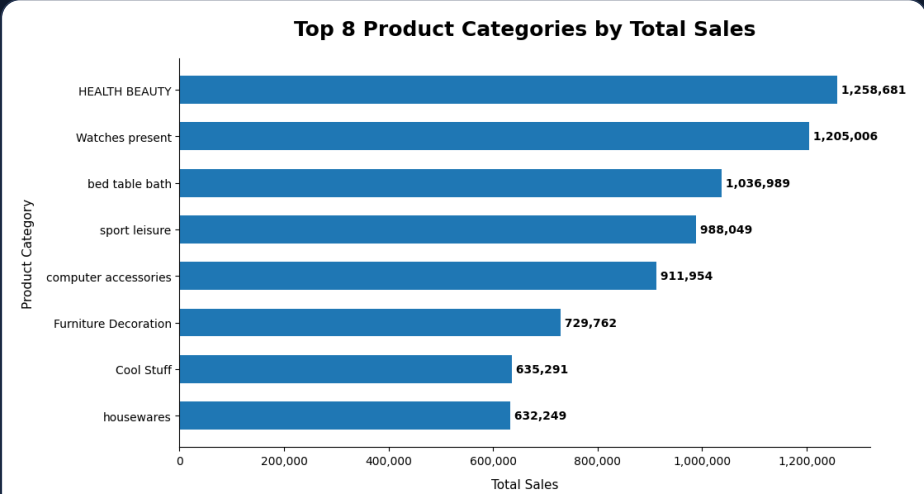
print("Top 8 product categories by total sales:")
print(top_8_categories)
```

SQL OUTPUT

	product_category character varying (100)	total_sales numeric
1	HEALTH BEAUTY	1258681.34
2	Watches present	1205005.68
3	bed table bath	1036988.68
4	sport leisure	988048.97
5	computer accessories	911954.32
6	Furniture Decoration	729762.49

Total rows: 74 | Query complete 00:00:00.584

PYTHON OUTPUT / VISUALIZATION



Source-faithful analytical code shown; visualization output is preserved from the notebook.

QUESTION 04 / 15

BASIC ANALYTICS

Calculate the percentage of orders that were paid in installments.

QUERY.SQL

```sql
-- Calculate the percentage of orders that were paid in installments.

SELECT
    ROUND(
        100.0 * COUNT(DISTINCT CASE
            WHEN payment_installments > 1 THEN order_id
        END)
        / COUNT(DISTINCT order_id),
        2
    ) AS percentage_orders_paid_in_installments
FROM payments;
```

ANALYSIS.PY

```python
# BASIC PROBLEM 4
# Calculate the percentage of orders that were paid
# in installments.

installment_orders = payments[
    payments["payment_installments"] > 1
]["order_id"].nunique()

total_orders = payments["order_id"].nunique()

installment_percentage = (
    installment_orders / total_orders
) * 100

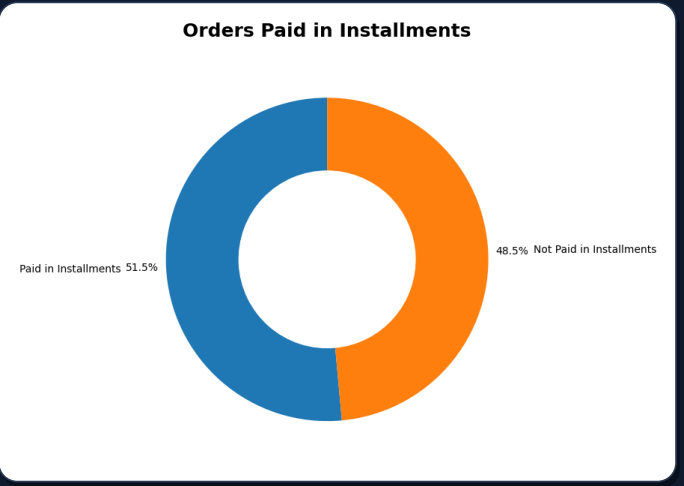
non_installment_percentage = 100 - installment_percentage

print(
    f"Percentage of orders paid in installments: "
    f"{installment_percentage:.2f}%"
)
```

SQL OUTPUT

	percentage_orders_paid_in_installments numeric
1	51.46

PYTHON OUTPUT / VISUALIZATION



Source-faithful analytical code shown; visualization output is preserved from the notebook.

QUESTION 05 / 15

Count the number of customers from each state.

BASIC ANALYTICS

QUERY.SQL

```sql
-- Count the number of customers from each state.

SELECT
    customer_state,
    COUNT(*) AS customer_count
FROM customers
GROUP BY customer_state
ORDER BY customer_count DESC;
```

ANALYSIS.PY

```python
# BASIC PROBLEM 5
# Count the number of customers from each state.

customers_by_state = (
    customers
    .groupby("customer_state")
    .size()
    .reset_index(name="customer_count")
    .sort_values("customer_count", ascending=True)
)

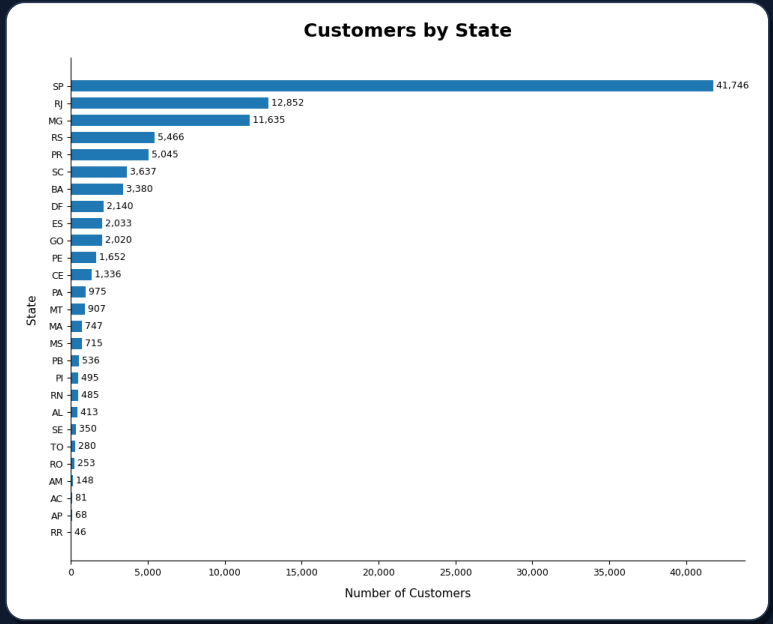
print("Number of customers from each state:")
print(
    customers_by_state.sort_values(
        "customer_count",
        ascending=False
    )
)
```

SQL OUTPUT

	customer_state character varying (10)	customer_count bigint
1	SP	41746
2	RJ	12852
3	MG	11635
4	RS	5466
5	PR	5045
6	SC	3637

Total rows: 27 | Query complete 00:00:00.200

PYTHON OUTPUT / VISUALIZATION



Source-faithful analytical code shown; visualization output is preserved from the notebook.

QUESTION 06 / 15

Calculate the number of orders per month in 2018.

INTERMEDIATE ANALYTICS

QUERY.SQL

```sql
-- Calculate the number of orders per month in 2018.

SELECT
    EXTRACT(MONTH FROM order_purchase_timestamp) AS month,
    COUNT(*) AS order_count
FROM orders
WHERE order_purchase_timestamp >= '2018-01-01'
  AND order_purchase_timestamp < '2019-01-01'
GROUP BY EXTRACT(MONTH FROM order_purchase_timestamp)
ORDER BY month;
```

ANALYSIS.PY

```python
# INTERMEDIATE PROBLEM 1
# Calculate the number of orders per month in 2018.

orders_2018 = orders[
    (orders["order_purchase_timestamp"] >= "2018-01-01") &
    (orders["order_purchase_timestamp"] < "2019-01-01")
].copy()

orders_2018["month"] = orders_2018[
    "order_purchase_timestamp"
].dt.month

monthly_orders_2018 = (
    orders_2018
    .groupby("month")
    .size()
    .reset_index(name="order_count")
)

# Keep all 12 months in order
monthly_orders_2018 = monthly_orders_2018.sort_values("month")

# Month names
month_names = [
    "Jan", "Feb", "Mar", "Apr", "May", "Jun",
    "Jul", "Aug", "Sep", "Oct", "Nov", "Dec"
]

monthly_orders_2018["month_name"] = monthly_orders_2018[
    "month"
].apply(lambda x: month_names[x - 1])

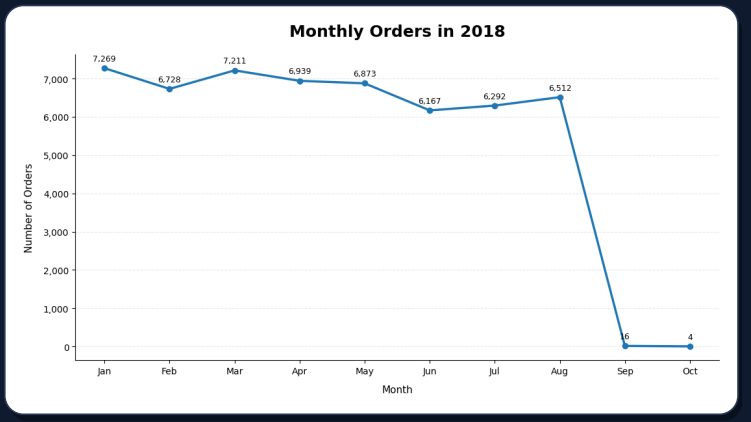
print("Number of orders per month in 2018:")
print(
    monthly_orders_2018[
        ["month_name", "order_count"]
    ]
)
```

SQL OUTPUT

	month numeric	order_count bigint
1	1	7269
2	2	6728
3	3	7211
4	4	6939
5	5	6873
6	6	6167

Total rows: 10 | Query comple

PYTHON OUTPUT / VISUALIZATION



Source-faithful analytical code shown; visualization output is preserved from the notebook.

QUESTION 07 / 15

Find the average number of products per order, grouped by customer city.

INTERMEDIATE ANALYTICS

QUERY.SQL

```sql
-- Find the average number of products per order,
-- grouped by customer city.

WITH products_per_order AS (
    SELECT
        o.order_id,
        o.customer_id,
        COUNT(oi.order_item_id) AS product_count
    FROM orders o
    JOIN order_items oi
        ON o.order_id = oi.order_id
    GROUP BY o.order_id, o.customer_id
)

SELECT
    c.customer_city,
    AVG(ppo.product_count) AS average_products_per_order
FROM products_per_order ppo
JOIN customers c
    ON ppo.customer_id = c.customer_id
GROUP BY c.customer_city
ORDER BY average_products_per_order DESC;
```

ANALYSIS.PY

```python
# INTERMEDIATE PROBLEM 2
# Find the average number of products per order,
# grouped by customer city.

# Count products in each order
products_per_order = (
    order_items
    .groupby("order_id")
    .size()
    .reset_index(name="product_count")
)

# Connect orders with customer city
order_city_products = (
    orders[["order_id", "customer_id"]]
    .merge(
        products_per_order,
        on="order_id",
        how="inner"
    )
    .merge(
        customers[["customer_id", "customer_city"]],
        on="customer_id",
        how="inner"
    )
)

# Calculate average products per order by city
average_products_by_city = (
    order_city_products
    .groupby("customer_city", as_index=False)["product_count"]
    .mean()
    .rename(columns={
        "product_count": "average_products_per_order"
    })
    .sort_values(
        "average_products_per_order",
        ascending=False
    )
)

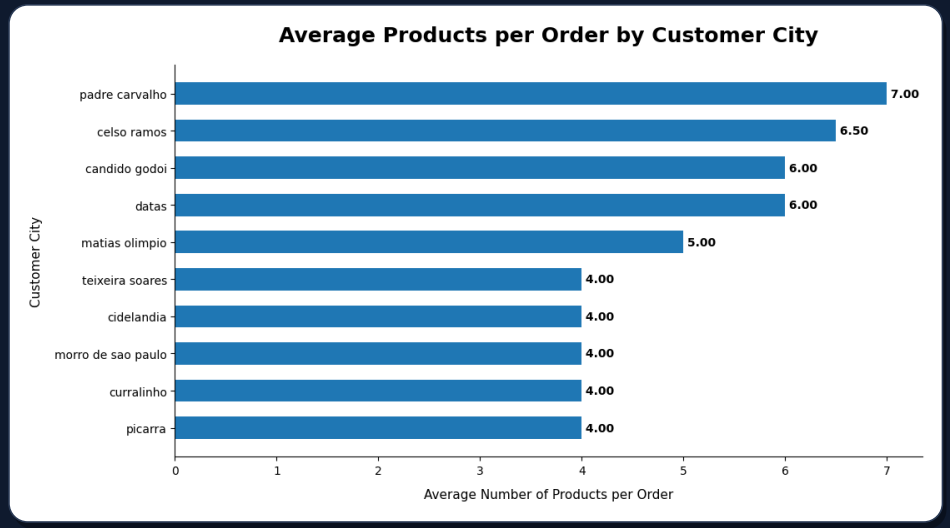
print("Average number of products per order by customer city:")
print(average_products_by_city)
```

SQL OUTPUT

	customer_city character varying (100)	average_products_per_order numeric
1	padre carvalho	7.0000000000000000
2	celso ramos	6.5000000000000000
3	candido godoi	6.0000000000000000
4	datas	6.0000000000000000
5	matias olimpio	5.0000000000000000
6	morro de sao paulo	4.0000000000000000

Total rows: 4110 | Query complete 00:00:01.399

PYTHON OUTPUT / VISUALIZATION



Source-faithful analytical code shown; visualization output is preserved from the notebook.

Calculate the percentage of total revenue contributed by each product category.

QUERY.SQL

```sql
-- Calculate the percentage of total revenue contributed
-- by each product category.

WITH category_revenue AS (
    SELECT
        p."product category" AS product_category,
        SUM(oi.price) AS revenue
    FROM order_items oi
    JOIN products p
        ON oi.product_id = p.product_id
    WHERE p."product category" IS NOT NULL
    GROUP BY p."product category"
)

SELECT
    product_category,
    revenue,
    ROUND(
        100.0 * revenue / SUM(revenue) OVER (),
        2
    ) AS revenue_percentage
FROM category_revenue
ORDER BY revenue DESC;
```

ANALYSIS.PY

```python
# INTERMEDIATE PROBLEM 3
# Calculate the percentage of total revenue contributed
# by each product category.

# Calculate revenue by product category
category_revenue = (
    order_items
    .merge(
        products[["product_id", "product category"]],
        on="product_id",
        how="inner"
    )
    .groupby("product category", as_index=False)["price"]
    .sum()
    .rename(columns={
        "product category": "product_category",
        "price": "revenue"
    })
)

# Calculate percentage of total revenue
total_revenue = category_revenue["revenue"].sum()

category_revenue["revenue_percentage"] = (
    category_revenue["revenue"] / total_revenue
) * 100

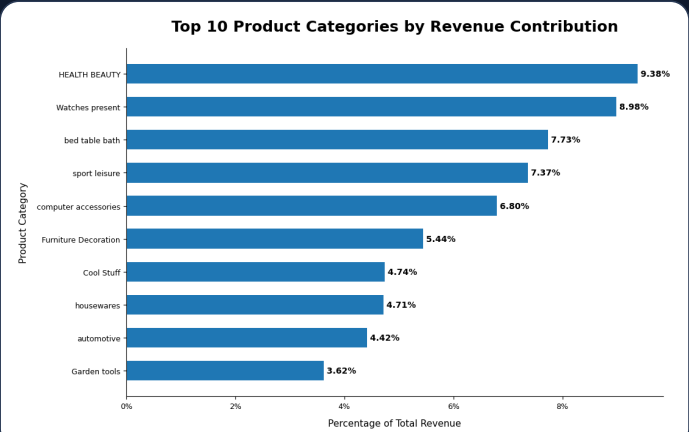
category_revenue = category_revenue.sort_values(
    "revenue_percentage",
    ascending=False
)

print("Percentage of total revenue contributed by each category:")
print(category_revenue)
```

SQL OUTPUT

	product_category character varying (100)	revenue numeric	revenue_percentage numeric
1	HEALTH BEAUTY	1258681.34	9.38
2	Watches present	1205005.68	8.98
3	bed table bath	1036988.68	7.73
4	sport leisure	988048.97	7.37
5	computer accessories	911954.32	6.80

PYTHON OUTPUT / VISUALIZATION



Source-faithful analytical code shown; visualization output is preserved from the notebook.

QUESTION 09 / 15

INTERMEDIATE ANALYTICS

Identify the correlation between product price and the number of times a product has been purchased.

QUERY.SQL

```sql
-- Identify the correlation between product price and the
-- number of times a product has been purchased.

WITH product_purchase_data AS (
    SELECT
        product_id,
        AVG(price) AS product_price,
        COUNT(*) AS times_purchased
    FROM order_items
    GROUP BY product_id
)

SELECT
    CORR(product_price, times_purchased) AS correlation
FROM product_purchase_data;
```

ANALYSIS.PY

```python
# INTERMEDIATE PROBLEM 4
# Identify the correlation between product price and the
# number of times a product has been purchased.

# Calculate average product price and purchase frequency
product_purchase_data = (
    order_items
    .groupby("product_id")
    .agg(
        product_price=("price", "mean"),
        times_purchased=("product_id", "count")
    )
    .reset_index()
)

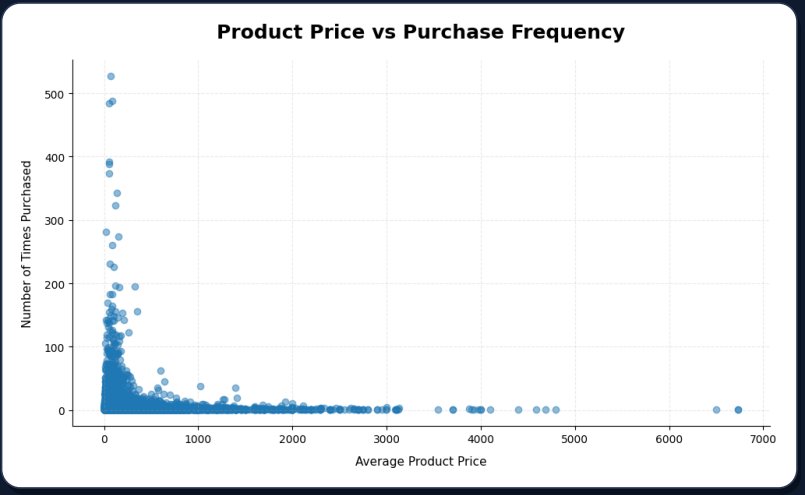
# Calculate correlation
correlation = product_purchase_data[
    ["product_price", "times_purchased"]
].corr().iloc[0, 1]

print(
    f"Correlation between product price and "
    f"number of times purchased: {correlation:.4f}"
)
```

SQL OUTPUT

	correlation double precision
1	-0.032139862680947075

PYTHON OUTPUT / VISUALIZATION



Source-faithful analytical code shown; visualization output is preserved from the notebook.

QUESTION 10 / 15

INTERMEDIATE ANALYTICS

Calculate the total revenue generated by each seller, and rank them by revenue.

QUERY.SQL

```sql
-- Calculate the total revenue generated by each seller,
-- and rank them by revenue.

WITH seller_revenue AS (
    SELECT
        seller_id,
        SUM(price) AS total_revenue
    FROM order_items
    GROUP BY seller_id
)

SELECT
    seller_id,
    total_revenue,
    RANK() OVER (
        ORDER BY total_revenue DESC
    ) AS revenue_rank
FROM seller_revenue
ORDER BY revenue_rank;
```

ANALYSIS.PY

```python
# INTERMEDIATE PROBLEM 5
# Calculate the total revenue generated by each seller,
# and rank them by revenue.

# Calculate total revenue generated by each seller
seller_revenue = (
    order_items
    .groupby("seller_id", as_index=False)["price"]
    .sum()
    .rename(columns={
        "price": "total_revenue"
    })
    .sort_values(
        "total_revenue",
        ascending=False
    )
)

# Rank sellers by revenue
seller_revenue["revenue_rank"] = (
    seller_revenue["total_revenue"]
    .rank(
        method="min",
        ascending=False
    )
    .astype(int)
)

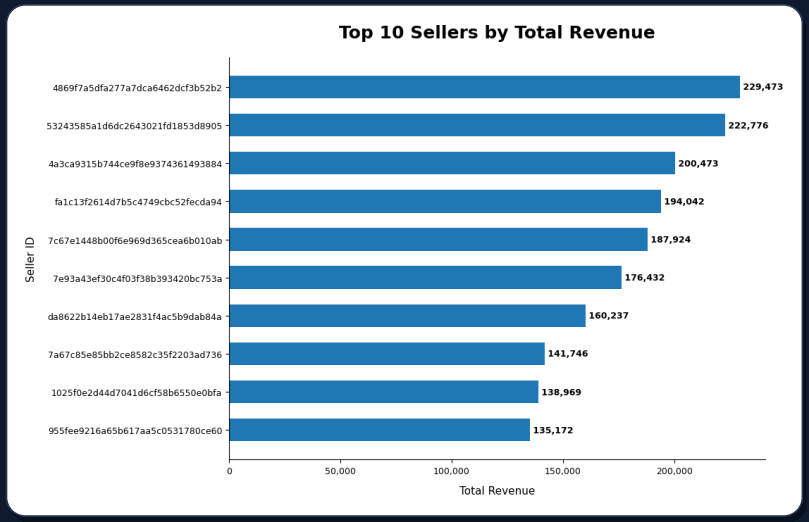
print("Seller revenue ranking:")
print(seller_revenue)
```

SQL OUTPUT

	seller_id character varying (50)	total_revenue numeric	revenue_rank bigint
1	4869f7a5dfa277a7dca6462dcf3b52…	229472.63	1
2	53243585a1d6dc2643021fd1853d8…	222776.05	2
3	4a3ca9315b744ce9f8e93743614938…	200472.92	3
4	fa1c13f2614d7b5c4749cbc52fecda94	194042.03	4
5	7c67e1448b00f6e969d365cea6b010…	187923.89	5
6	7e93a43ef30c4f03f38b393420bc75…	176431.87	6

Total rows: 3095 | Query complete 00:00:00.456

PYTHON OUTPUT / VISUALIZATION



Source-faithful analytical code shown; visualization output is preserved from the notebook.

QUESTION 11 / 15

ADVANCED ANALYTICS

Calculate the moving average of order values for each customer over their order history.

QUERY.SQL

```sql
-- Calculate the moving average of order values for each
-- customer over their order history.

WITH order_values AS (
    SELECT
        o.order_id,
        o.customer_id,
        o.order_purchase_timestamp,
        SUM(oi.price) AS order_value
    FROM orders o
    JOIN order_items oi
        ON o.order_id = oi.order_id
    GROUP BY
        o.order_id,
        o.customer_id,
        o.order_purchase_timestamp
)

SELECT
    order_id,
    customer_id,
    order_purchase_timestamp,
    order_value,
    AVG(order_value) OVER (
        PARTITION BY customer_id
        ORDER BY order_purchase_timestamp
        ROWS BETWEEN 1 PRECEDING AND CURRENT ROW
    ) AS moving_average
FROM order_values
ORDER BY customer_id, order_purchase_timestamp;
```

ANALYSIS.PY

```python
# ADVANCED PROBLEM 1
# Calculate the moving average of order values for each
# customer over their order history.

# Add unique customer identifier to orders
orders_with_customer = (
    orders[
        ["order_id", "customer_id", "order_purchase_timestamp"]
    ]
    .merge(
        customers[["customer_id", "customer_unique_id"]],
        on="customer_id",
# … middle analytical lines omitted on this slide …
    )["price"]
    .sum()
    .rename(columns={
        "price": "order_value"
    })
)

# Sort orders by customer and purchase date
order_values = order_values.sort_values(
    ["customer_unique_id", "order_purchase_timestamp"]
)

# Calculate 2-record moving average
order_values["moving_average"] = (
    order_values
    .groupby("customer_unique_id")["order_value"]
    .rolling(window=2)
    .mean()
    .reset_index(level=0, drop=True)
)

print("Moving average of order values for each customer:")
print(order_values)
```

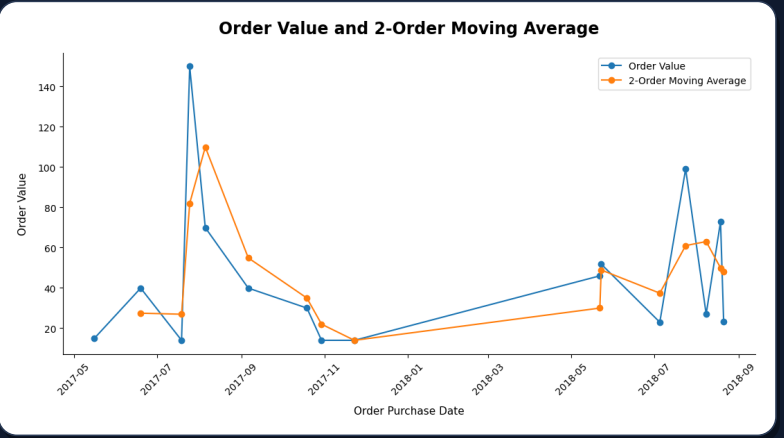
SQL OUTPUT

Showing rows: 1 to 1000 Page No: 1 of 99

	order_id character varying (50)	customer_id character varying (50)	order_purchase_timestamp timestamp without time zone	order_value numeric	moving_average numeric
1	5f79b5b0931d63f1a42989eb65b9d…	00012a2ce6f8dcda20d059ce98491…	2017-11-14 16:08:26	89.80	89.8000000000000000
2	a44895d095d7e0702b6a162fa2dbe…	000161a058600d5901f007fab4c271…	2017-07-16 09:40:32	54.90	54.9000000000000000
3	316a104623542e4d75189bb372bc5…	0001fd6190edaaf884bcaf3d49edf079	2017-02-28 11:06:43	179.99	179.9900000000000000
4	5825ce2e88d5346438686b0bba99e…	0002414f95344307404f0ace7a26f1…	2017-08-16 13:09:20	149.90	149.9000000000000000
5	0ab7fb08086d4af91414 53c91878ed…	000379cdec625522490c315e70c7a…	2018-04-02 13:42:17	93.00	93.0000000000000000
6	cd3558a10d854487b4f907e9b326a…	0004164d20a9e969af783496f34086…	2017-04-12 08:35:12	59.99	59.9900000000000000

Total rows: 98666 Query complete 00:00:02.893 CRLF

PYTHON OUTPUT / VISUALIZATION



Python code abbreviated where marked for slide fit; analytical source lines shown are unchanged.

QUESTION 12 / 15

Calculate the cumulative sales per month for each year.

ADVANCED ANALYTICS

QUERY.SQL

```sql
-- Calculate the cumulative sales per month for each year.

WITH monthly_sales AS (
    SELECT
        EXTRACT(YEAR FROM o.order_purchase_timestamp) AS year,
        EXTRACT(MONTH FROM o.order_purchase_timestamp) AS month,
        SUM(oi.price) AS monthly_sales
    FROM orders o
    JOIN order_items oi
        ON o.order_id = oi.order_id
    GROUP BY
        EXTRACT(YEAR FROM o.order_purchase_timestamp),
        EXTRACT(MONTH FROM o.order_purchase_timestamp)
)

SELECT
    year,
    month,
    monthly_sales,
    SUM(monthly_sales) OVER (
        PARTITION BY year
        ORDER BY month
    ) AS cumulative_sales
FROM monthly_sales
ORDER BY year, month;
```

ANALYSIS.PY

```python
# ADVANCED PROBLEM 2
# Calculate the cumulative sales per month for each year.

# Calculate monthly sales
monthly_sales = (
    orders[
        ["order_id", "order_purchase_timestamp"]
    ]
    .merge(
        order_items[["order_id", "price"]],
        on="order_id",
        how="inner"
    )
)

monthly_sales["year"] = (
    monthly_sales["order_purchase_timestamp"].dt.year
)

monthly_sales["month"] = (
    monthly_sales["order_purchase_timestamp"].dt.month
)

monthly_sales = (
    monthly_sales
    .groupby(["year", "month"], as_index=False)["price"]
    .sum()
    .rename(columns={
        "price": "monthly_sales"
    })
    .sort_values(["year", "month"])
)

# Calculate cumulative sales for each year
monthly_sales["cumulative_sales"] = (
    monthly_sales
    .groupby("year")["monthly_sales"]
    .cumsum()
)

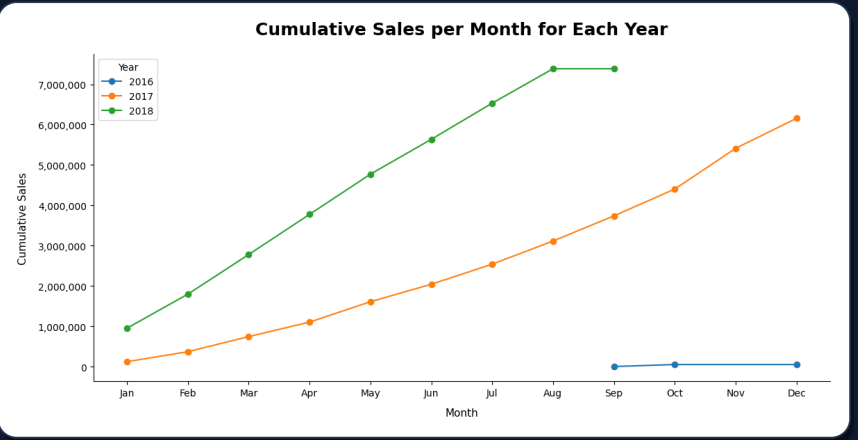
print("Cumulative sales per month for each year:")
print(monthly_sales)
```

SQL OUTPUT

	year numeric	month numeric	monthly_sales numeric	cumulative_sales numeric
1	2016	9	267.36	267.36
2	2016	10	49507.66	49775.02
3	2016	12	10.90	49785.92
4	2017	1	120312.87	120312.87
5	2017	2	247303.02	367615.89
6	2017	3	374344.30	741960.19

Total rows: 24 | Query complete 00:00:00.866

PYTHON OUTPUT / VISUALIZATION



Source-faithful analytical code shown; visualization output is preserved from the notebook.

QUESTION 13 / 15

Calculate the year-over-year growth rate of total sales.

ADVANCED ANALYTICS

QUERY.SQL

```sql
-- Calculate the year-over-year growth rate of total sales.

WITH yearly_sales AS (
    SELECT
        EXTRACT(YEAR FROM o.order_purchase_timestamp) AS year,
        SUM(oi.price) AS total_sales
    FROM orders o
    JOIN order_items oi
        ON o.order_id = oi.order_id
    GROUP BY EXTRACT(YEAR FROM o.order_purchase_timestamp)
),

yearly_sales_with_previous AS (
    SELECT
        year,
        total_sales,
        LAG(total_sales) OVER (
            ORDER BY year
        ) AS previous_year_sales
    FROM yearly_sales
)

SELECT
    year,
    total_sales,
    previous_year_sales,
    ROUND(
        100.0 * (total_sales - previous_year_sales)
        / NULLIF(previous_year_sales, 0),
        2
    ) AS yoy_growth_rate
FROM yearly_sales_with_previous
ORDER BY year;
```

ANALYSIS.PY

```python
# ADVANCED PROBLEM 3
# Calculate the year-over-year growth rate of total sales.

# Calculate total sales for each year
yearly_sales = (
    orders[
        ["order_id", "order_purchase_timestamp"]
    ]
    .merge(
        order_items[["order_id", "price"]],
        on="order_id",
        how="inner"
    )
)

yearly_sales["year"] = (
    yearly_sales["order_purchase_timestamp"].dt.year
)

yearly_sales = (
    yearly_sales
    .groupby("year", as_index=False)["price"]
    .sum()
    .rename(columns={
        "price": "total_sales"
    })
    .sort_values("year")
)

# Calculate previous year's sales
yearly_sales["previous_year_sales"] = (
    yearly_sales["total_sales"].shift(1)
)

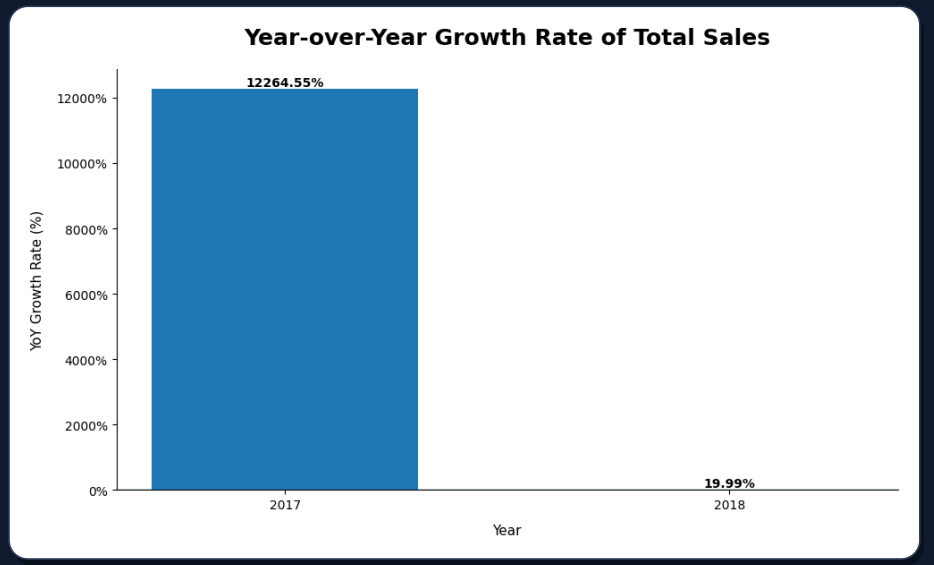
# Calculate year-over-year growth rate
yearly_sales["yoy_growth_rate"] = (
    (
        yearly_sales["total_sales"]
        - yearly_sales["previous_year_sales"]
    )
    / yearly_sales["previous_year_sales"]
) * 100

print("Year-over-Year Growth Rate of Total Sales:")
print(yearly_sales)
```

SQL OUTPUT

	year numeric	total_sales numeric	previous_year_sales numeric	yoy_growth_rate numeric
1	2016	49785.92	[null]	[null]
2	2017	6155806.98	49785.92	12264.55
3	2018	7386050.80	6155806.98	19.99

PYTHON OUTPUT / VISUALIZATION



Source-faithful analytical code shown; visualization output is preserved from the notebook.

ADVANCED ANALYTICS

Calculate the retention rate of customers, defined as the percentage of customers who make another purchase within 6 months of their first purchase.

QUERY.SQL

```sql
-- Retention within 6 months using customer_id
WITH first_purchase AS (
    SELECT customer_id, MIN(order_purchase_timestamp) AS first_date
    FROM orders GROUP BY customer_id
), retained AS (
    SELECT DISTINCT f.customer_id
    FROM first_purchase f JOIN orders o
        ON o.customer_id = f.customer_id
    WHERE o.order_purchase_timestamp > f.first_date
      AND o.order_purchase_timestamp <= f.first_date + INTERVAL '6 months'
)
SELECT
    COUNT(f.customer_id) AS total_customers,
    COUNT(r.customer_id) AS retained_customers,
    ROUND(
        100.0 * COUNT(r.customer_id)
        / NULLIF(COUNT(f.customer_id), 0), 2
    ) AS retention_rate
FROM first_purchase f
LEFT JOIN retained r ON r.customer_id = f.customer_id;
```

ANALYSIS.PY

```python
# Retention analysis using customer_id
first_purchase = (
    orders.groupby('customer_id', as_index=False)
    ['order_purchase_timestamp'].min()
    .rename(columns={'order_purchase_timestamp': 'first_date'})
)
orders_with_first = orders.merge(first_purchase, on='customer_id')
retained = orders_with_first[
    (
        orders_with_first['order_purchase_timestamp']
        > orders_with_first['first_date']
    ) &
    (
        orders_with_first['order_purchase_timestamp']
        <= orders_with_first['first_date'] + pd.DateOffset(months=6)
    )
]['customer_id'].nunique()
total = first_purchase['customer_id'].nunique()
rate = round(retained / total * 100, 2) if total else 0
# Reported output: 0 retained; 0.00% retention
```

SQL OUTPUT

	total_customers	retained_customers	retention_rate
1	99441	0	0.00

Query result · 1 row

PYTHON OUTPUT / VISUALIZATION

```
Total Customers: 99,441
Retained Customers: 0
Not Retained Customers: 99,441
Retention Rate: 0.00%
```



Source-faithful analytical code shown; visualization output is preserved from the notebook.

QUESTION 15 / 15

Identify the top 3 customers who spent the most money in each year.

ADVANCED ANALYTICS

QUERY.SQL

```sql
-- Identify the top 3 customers who spent the most money
-- in each year.

WITH customer_yearly_spending AS (
    SELECT
        o.customer_id,
        EXTRACT(YEAR FROM o.order_purchase_timestamp) AS year,
        SUM(oi.price) AS total_spent
    FROM orders o
    JOIN order_items oi
        ON o.order_id = oi.order_id
    GROUP BY
        o.customer_id,
        EXTRACT(YEAR FROM o.order_purchase_timestamp)
),

ranked_customers AS (
    SELECT
        customer_id,
        year,
        total_spent,
        RANK() OVER (
            PARTITION BY year
            ORDER BY total_spent DESC
        ) AS spending_rank
    FROM customer_yearly_spending
)

SELECT
    year,
    customer_id,
    total_spent,
    spending_rank
FROM ranked_customers
WHERE spending_rank <= 3
ORDER BY year, spending_rank;
```

ANALYSIS.PY

```python
# ADVANCED PROBLEM 5
# Identify the top 3 customers who spent the most money
# in each year.

# Merge orders with customer information
# to get the unique customer identifier
customer_order_data = (
    orders[
        [
            "order_id",
            "customer_id",
            "order_purchase_timestamp"
# … middle analytical lines omitted on this slide …

# Keep top 3 customers for each year
top_3_customers = (
    customer_yearly_spending[
        customer_yearly_spending["spending_rank"] <= 3
    ]
    .sort_values(
        [
            "year",
            "spending_rank"
        ]
    )
    .reset_index(drop=True)
)

# Convert rank to integer
top_3_customers["spending_rank"] = (
    top_3_customers["spending_rank"]
    .astype(int)
)

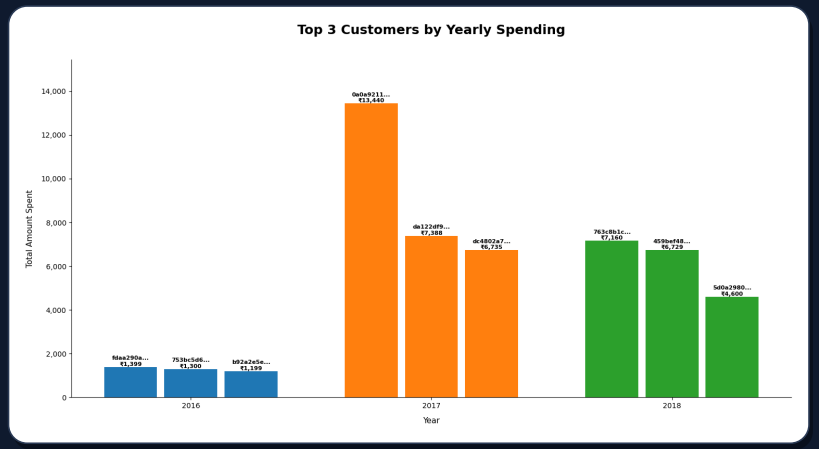
print("Top 3 customers by yearly spending:")
print(top_3_customers)
```

SQL OUTPUT

	year numeric	customer_id character varying (50)	total_spent numeric	spending_rank bigint
1	2016	a9dc96b027d1252bbac0a9b72d837…	1399.00	1
2	2016	1d34ed25963d5aae4cf3d7f3a4cda1…	1299.99	2
3	2016	4a06381959b6670756de02e07b838…	1199.00	3
4	2017	1617b1357756262bfa56ab541c47b…	13440.00	1
5	2017	c6e2731c5b391845f6800c97401a4…	6735.00	2
6	2017	3fd6777bbce08a352fddd04e4a7cc8…	6499.00	3

Total rows: 9 | Query complete 00:00:02.320

PYTHON OUTPUT / VISUALIZATION



Python code abbreviated where marked for slide fit; analytical source lines shown are unchanged.